CMSI 3802

Quiz 1 Preparation

When

The exam opens at the end of class on February 24, 2025 and closes 20 minutes before the start of class on February 26, 2025.

Scope

Computation, Language Design, Ohm, Compilers and Interpreters, Setting up complex projects with Node.js, unit tests, external libraries, and c8.

Ground Rules

You will take the exam on BrightSpace. There is 90 minute time limit (135 with those having time-and-a-half DSS accommodations). You **MAY** use books, notes, and web *searches* to look things up. You will not be spied on: there is no browser lock down and hence no need to hide a mobile device in a bag of potato chips. However, you **MAY NOT SOLICIT** answers in any way. That is, there is to be no asking for help, no posting on forums, no communication with other humans or intelligent bots in any way; you can only "look things up" but you may never "ask." You also **MAY NOT** post answers or help any other test taker either. You are bound by an honor code to follow these rules.

You have to put in the time for effortful self-study. Although the exam is open resources, you will not have time to look *everything* up. Those who come in with a strong comfort level with the material will finish on time. I am assessing your fluency and your proficiency with the material, not your Google-Fu.

The exam becomes available at 13:30 America/Los Angeles time on February 24 and must be submitted by 11:30 America/Los Angeles time on February 26. You may complete the exam within any 90 (or 135) minute period in that window. Make sure to submit the exam before time runs out.

Preparation Checklist

Do each of the following to maximize your preparation:

- ✅ Attend the scheduled lectures. Review the recordings for class periods you missed. Review the recordings for portions of the class sessions that you feel you need to review.
- ✅ Read and engage with all of the course notes within the scope of the exam.
- ✅ Do the assigned readings. Reading *both* the course notes and outside materials is empowering: you are seeing the same material presented in two different formats.
- ✅ Follow some of the links within the course notes to outside articles and videos. This gives you access to new material as well as a third presentation of the course content.
- ✅ Do the first two homework assignments. If you worked in a team, you are still responsible for understanding all of the solutions. If you were irresponsible enough to "split the work" so that you learned nothing at all about work that your teammate did, then you are not living up to your responsibilities as a student. Ask your partner to explain their work to you.
- ✅ Review and understand my solutions to the first two assignments.
- ✅ DO THE RECALL QUESTIONS IN THE NOTES PERIODICALLY. Active recall is the most powerful way to move knowledge from working memory to long-term memory.
- ✅ Take the practice exam on BrightSpace.

Were you able to check off every box?

Things to Know

Some big picture things:

- Some Computation and Computer Science History
- How to write Ohm grammars
- Difference between capitalized and non-capitalized categories
- How to give a parse tree for a given derivation in a grammar
- What it means for a grammar to be ambiguous

- How to read and write grammars for simple formal languages and for little programming languages
- Going between prose descriptions of languages and their grammars (both ways)
- The difference between lexical analysis and parsing
- What tokens are
- The difference between lexical and phrase rules
- Differences between CST and AST and how to draw them
- What the frontier of the CST is made up of
- A lot about ambiguity
- A lot about precedence
- Associativity (L, R, None, or indeterminate)
- The use of non-associativity, esp. in relational operators
- How can we make `-2**2` illegal (like it is in JS)
- How to write an Ohm grammar
- Ordering of rules in Ohm ("big" things first)
- Using Ohm: `ohm.grammar`, `ohm.createSemantics`
- How to draw an abbreviated AST
- How to draw an internal representation of an AST
- The difference between a compiler and an interpreter
- The phases of a traditional compiler
- How modern compilers are structured (Anders Hejlsberg video)
- AOT vs JIT
- What is syntax, what is static semantics
- Given an English description of a language figure out which checks are syntactic vs semantic
- Given an Ohm grammar, explain what it is saying (at a high level)
- How to write an AST for sophisticated JavaScript

An Outline of Course Content

The fact that active recall is better for acquiring long-term knowledge does not mean that outlines and concept maps are not useful. Learners should use multiple techniques—think "both and" rather than "either or."

```
Computation and its History
    Definitions of Information, Computation, and Language
    Computation is the study of Languages and Automata
    To many, computing is pop culture
    But it has a history
```

```
        Ancient times
        Numerals
        Early computing recipes
        Early computing machines
        Ada Lovelace
        Logicism vs Formalism vs Intuitionism
        Hilbert's Program
        Russell and Whitehead
        Lambda Calculus, Gödel's Recusive Functions, Turing Machines
        Alan Turing's paper
            Formalized effective computation
            Church-Turing Thesis
            Computational universality
```

Format of the Exam

An ideal assessment of your understanding of programming language concepts would examine your fluency with the course material through an oral exam. However, given the the number of students, this assessment will be on your recognition and evaluation of concept expression in the form of multiple choice and similar questions.

At some point during the exam window, find the quiz on BrightSpace and take it. You will have 90 minutes to complete it. Upon submission you should receive feedback and your score will be recorded in the grade book.

About the Problems

There will be 20 questions. Some will be multiple choice, some multi-select, and some matching. There are no free-form answers, so the exam will be autograded and you will see your score immediately after it is submitted.

Here are the general areas of coverage for each problem. Don't read too much into these hints or you will end up getting confused. The hints are supposed to help you. If you don't understand the hint, let it go—do not ask for clarification (since the hints were not required to be given in the first place).

- Ada Lovelace
- Alan Turing's paper
- Tokens
- An AST

- An AST
- While Statements in Ohm
- An expression that Java and JavaScript do not parse
- The `|>` operator
- How to state rules so that something works the way you intend
- Importance of parentheses in transpiling
- Checking that a return statement appears
- Lexical vs Phrase categories
- Something about an operation in Ohm
- Precedence and Associativity
- Given an Ohm grammar, what language is it specifying
- Which of the following statements are true about this grammar
- Something from a video on the language design notes
- Consequences of a particular syntax design decision
- CSTs vs ASTs
- High level languages, interpreters, and compilers

The problem order will be randomized for each student.